

BRAC University
Department of Computer Science and Engineering

Examination: Quiz 04
Duration: 35 Minutes

Semester: Summer 26
Full Marks: 15

CSE420: Compiler Design

1. Explain with an example how a compiler converts member functions of classes from object-oriented programming to global functions. (2)
2. Consider the following SDT. Draw the Annotated Parse Tree and evaluate the tree along with the three address code for the String: (13)

for (f=4; f!=0; f--)

```
func temp() {
    temp +=1;
    return temp;
}
```

```
int newlabel() {
    p = p + 1;
    return "L" || p;
}
```

SDT:

P → {*p* = 10;} {*temp* = 4;} **S**

S → **id** = **E** {*tac* = *st.pop()*; *tac* = *tac* || *gen(id.lexeme* '=*' t' temp*);
st.push(tac); *S.code* = *st.pop()*; }

S → **id** **addop** { if (*addop* == "++") {
tac = *gen('t' temp* '=' *id.lexeme* ' + *I'*); *tac* = *tac* || *gen(id.lexeme* '=' ' *t' temp*);
temp = *temp()*; *S.code* = *tac*; }
} else {
tac = *gen('t' temp* '=' *id.lexeme* ' - *I'*); *tac* = *tac* || *gen(id.lexeme* '=' ' *t' temp*);
temp = *temp()*; *S.code* = *tac*; }
}

S → **for** ({ *temp* = *temp()*; }
S₁ ; { *temp* = *temp()*; }
B ; { *temp* = *temp()*; }
S₂) **S**₃
{ *test* = *newlabel()*; *inc* = *newlabel()*; *B.true* = *newlabel()*; *B.false* = *newlabel()*;
S.code = *S*₁.*code* || *label(test)* || *B.code* || *label(B.true)* || *S*₃.*code* || *label(inc)* || *S*₂.*code* || *gen('goto' test)*; }

S → ε {*S.code* = " ";}

B → **E**₁ **rel** **E**₂ {*B.code* = *E*₁.*code* || *gen('if' E*₁.*addr* *rel.op* *E*₂.*addr* 'goto' *B.true* 'goto' *B.false*); *st.pop()*;}

E → **E**₁ **arith** **E**₂ {*tac* = *st.pop()*; *t* = *gen('t' temp)*; *temp* = *temp()*; *add* = *gen('t' temp)*;
tac = *tac* || *gen(add* '=' *E*₁.*addr* *arith.op* *t*); *E.code* = *tac*; }

E → **id** {*E.addr* = *id.lexeme*; *E.code* = " ";}

E → **num** {*E.addr* = *num.val*; *st.push(gen('t' temp* '=' *num.val*));}

Name:	ID:	Section:
-------	-----	----------

BRAC University
Department of Computer Science and Engineering

Examination: Quiz 04

Duration: 35 Minutes

Semester: Summer 26

Full Marks: 15

CSE420: Compiler Design

1. Explain why an intermediate code generator does not need to specifically add any kind of instructions for access modifiers in the source code. (2)
2. Consider the following SDT. Draw the Annotated Parse Tree and evaluate the tree along with the three address code for the String: (13)

for (i=0; i<=10; i++)

```
func temp() {
    temp +=1;
    return temp;
}
```

```
int newlabel() {
    p = p - 1;
    return "L" || p;
}
```

SDT:

P → {*p = 10;*} **S**

S → **id** = **E** {*tac = st.pop(); tac = tac || gen(id.lexeme '=' t' temp);*
st.push(tac); S.code = st.pop(); }

B → **E**₁ **rel** **E**₂ {*B.code = E₁.code || gen('if' E₁.addr rel.op E₂.addr 'goto' B.true 'goto' B.false); st.pop();*}

E → **E**₁ **arith** **E**₂ {*tac = st.pop(); t = gen('t' temp); temp = temp(); add = gen('t' temp);*
tac = tac || gen(add '=' E₁.addr arith.op t); E.code = tac; }

| **id** {*E.addr = id.lexeme; E.code = " ";*}

| **num** {*E.addr = num.val; st.push(gen('t' temp '=' num.val));*}

S → **id** ++ {*tac = gen('t' temp '=' id.lexeme '+' 1); tac = tac || gen(id.lexeme '=' 't' temp);*
temp = temp(); S.code = tac; }

S → {*temp = 4;*} **for** ({ *temp = temp();*} **S**₁ ;
 { *temp = temp();*} **B** ; {*temp = temp();*} **S**₂) **S**₃
 { *test = newlabel(); inc = newlabel(); B.true = newlabel(); B.false = newlabel();*
S.code = S₁.code || label(test) || B.code || label(B.true) || S₃.code || label(inc) || S₂.code || gen('goto' test); }

S → ε {*S.code = " ";*}

Name:	ID:	Section:
-------	-----	----------

BRAC University
Department of Computer Science and Engineering

Examination: Quiz 04

Duration: 35 Minutes

Semester: Summer 26

Full Marks: 15

CSE420: Compiler Design

1. A student argues: "Since the compiler ignores visibility control during code generation, a hacker can simply modify the binary to access private fields of any class." Is this claim correct? What does this reveal about the nature of OOP safety guarantees — are they a runtime guarantee or a compile-time contract? (2)
2. Consider the following SDT. Draw the Annotated Parse Tree and evaluate the tree along with the three address code for the String: (13)

for (fo=8; fo>=0; fo++)

```
func temp() {
    temp = temp - 1;
    return temp;
}
```

```
int newlabel() {
    p = p + 1;
    return "L" || p;
}
```

SDT:

P → {*p* = 0;} {*temp* = 10;} **S**

B → **E**₁ **rel** **E**₂ {*B.code* = *E*₁.*code* || *gen('if' E*₁.*addr rel.op E*₂.*addr 'goto' B.true 'goto' B.false); st.pop();*}

E → **E**₁ **arith** **E**₂ {*tac* = *st.pop()*; *t* = *gen('t' temp)*; *temp* = *temp()*; *add* = *gen('t' temp)*;
tac = *tac* || *gen(add '=' E*₁.*addr arith.op t)*; *E.code* = *tac*; }

| **id** {*E.addr* = *id.lexeme*; *E.code* = " ";}

| **num** {*E.addr* = *num.val*; *st.push(gen('t' temp '=' num.val))*;} }

| **id = E** {*tac* = *st.pop()*; *tac* = *tac* || *gen(id.lexeme '=' t' temp)*;
st.push(tac); *S.code* = *st.pop()*; }

S → **id = E** {*tac* = *st.pop()*; *tac* = *tac* || *gen(id.lexeme '=' t' temp)*;
st.push(tac); *S.code* = *st.pop()*; }

S → ε {*S.code* = " "};}

| **id addop** { *if (addop == "++")* {
tac = *gen('t' temp '=' id.lexeme ' + 1')*; *tac* = *tac* || *gen(id.lexeme '=' 't' temp)*;
temp = *temp()*; *S.code* = *tac*; }

}
 else {
tac = *gen('t' temp '=' id.lexeme ' - 1')*; *tac* = *tac* || *gen(id.lexeme '=' 't' temp)*;
temp = *temp()*; *S.code* = *tac*; }

}

| **for ({temp = temp();} S₁ ; {temp = temp();}**

B ; {temp = temp();} S₂) S₃

{ *test* = *newlabel()*; *inc* = *newlabel()*; *B.true* = *newlabel()*; *B.false* = *newlabel()*;

S.code = *S*₁.*code* || *label(test)* || *B.code* || *label(B.true)* || *S*₃.*code* || *label(inc)* || *S*₂.*code* || *gen('goto' test)*; }

Name:	ID:	Section:
-------	-----	----------